

University Food Court Management System

Technical & Business Documentation

Project Information

SUPERVISORS

- Dr. Sarah Naiem
- Dr. Doaa Mohey Eldin

COURSE CODE

INF2201

Team Members

Name	Section	ID
Ahmed Mohamed Refaat	C2	692400154
Fady Fouad Milad	C3	692400508
Noura Maher Mohamed	C3	692400276
Ebrahim Yussif Ebrahim	C4	692400174
Ferial Mohamed Saad	C4	692400369
Noha Mohamed Hamdi	C3	692400012
Ahmed Yousry Tagelmoulouk	C3	692400615
Seif Eldin Mohamed	D1	692400535
Ahmed Said Abdelhaseb	C4	692400495
Mohammed Walid	A3	692400507
Ali Fahmi Abolela	B1	692400782

1. Project Overview

Purpose of the System

The **University Food Court Management System** is a robust, enterprise-grade relational database designed to manage the end-to-end operations of a dynamic university campus food court. It provides a centralized platform to handle vendors, diverse menus, dine-in table reservations, queue management, payments, and customer loyalty programs.

Main Objectives

- **Operational Efficiency:** Automate order processing, queue tracking, and payment verification to reduce wait times.
- **Customer Retention:** Implement a loyalty points system to encourage repeat purchases among students, professors, and staff.
- **Resource Management:** Dynamically manage physical dining tables to ensure optimal seating capacity.
- **Data Analytics:** Provide comprehensive business intelligence through advanced SQL querying to track revenue, popular items, and vendor performance.

Features of the System

- **Automated Queue Generation:** Seamlessly generates queue numbers and estimates wait times based on preparation times of ordered items.
- **Dynamic Table Tracking:** Automatically updates table statuses (Available, Occupied) based on active dine-in orders.
- **Integrated Loyalty Program:** Automatically awards points upon payment and allows redemption for discounts.
- **Robust Transaction Management:** Ensures data consistency using Stored Procedures and Triggers for order creation, cancellation, and payment processing.
- **Realistic Simulation & Analytics:** Pre-populated with rich sample data (70 users, 9 real-world vendors, hundreds of menu items, and 150 simulated orders) and equipped with 15 advanced analytical queries.

2. Database Design

The database is highly normalized to ensure data integrity, eliminate redundancy, and support complex relationships.

Explanation of all Tables

The system consists of 7 core tables: **Users**, **FoodVendors**, **MenuItems**, **FoodCourtTables**, **Orders**, **OrderDetails**, and **Payments**. They represent the core entities of a food service business.

Relationships between Tables

- **Users (1) to (M) Orders:** A user can place multiple orders.
- **FoodVendors (1) to (M) MenuItems:** A vendor offers multiple menu items.
- **MenuItems (1) to (M) OrderDetails:** A menu item can appear in many order details.
- **Orders (1) to (M) OrderDetails:** An order consists of multiple line items (details).
- **FoodCourtTables (1) to (M) Orders:** A table can host multiple dine-in orders over time.
- **Orders (1) to (1) Payments:** Each order has one unique payment record.

Keys and Constraints

- **Primary Keys (PK):** Every table features an auto-incrementing **IDENTITY(1,1)** integer Primary Key (e.g., **UserID**, **VendorID**, **OrderID**).
- **Foreign Keys (FK):** Used extensively to enforce referential integrity (e.g., **VendorID** in **MenuItems** references **FoodVendors**). **ON DELETE CASCADE** is utilized for **OrderDetails** and **Payments** when an **Order** is deleted.
- **Check Constraints:** Used to validate domain rules:
 - Positive values: **Price > 0**, **Quantity > 0**, **Amount >= 0**.
 - Status restrictions: **Role IN ('Student', 'Professor', 'Staff')**, **OrderStatus IN ('Pending', 'Preparing', 'Ready', 'Completed', 'Cancelled')**.

- **Unique Constraints:** Applied to **Email** in **Users**, **TableNumber** in **FoodCourtTables**, and **OrderID** in **Payments**.

ERD Explanation

The Entity-Relationship Model centers around the **Orders** table. **Orders** bridges the **Users** (who places the order), the **FoodCourtTables** (where they sit), and the **Payments** (how they pay). The **Orders** table breaks down its composite items into **OrderDetails**, which directly link back to **MenuItems** and, by extension, **FoodVendors**. This star-like schema facilitates highly efficient querying and reporting.

3. Tables Description

1. Users

- **Purpose:** Stores the profiles of all campus members interacting with the food court.
- **Important Columns:** `Role` (Student/Professor/Staff), `TotalPointsEarned`, `CurrentPointsBalance`.
- **Data Types:** `NVARCHAR` for strings, `INT` for points, `DATETIME` for timestamps.
- **Relationships:** PK `UserID`. Referenced by `Orders.UserID`.

2. FoodVendors

- **Purpose:** Catalogs the distinct food stalls and restaurants operating in the food court.
- **Important Columns:** `VendorName`, `Location`, `Status` (Active/Closed).
- **Data Types:** `NVARCHAR` for text.
- **Relationships:** PK `VendorID`. Referenced by `MenuItems.VendorID`.

3. MenuItems

- **Purpose:** Holds the complete menu for every vendor, including pricing and prep times.
- **Important Columns:** `ItemName`, `Category`, `Price`, `EstimatedPrepTime`.
- **Data Types:** `DECIMAL(10,2)` for financial values, `INT` for minutes.
- **Relationships:** PK `MenuID`, FK `VendorID`. Referenced by `OrderDetails.MenuID`.

4. FoodCourtTables

- **Purpose:** Manages the physical seating areas to assist with dine-in logistics.
- **Important Columns:** `TableNumber`, `AreaName`, `Capacity`, `Status`.
- **Data Types:** `INT` for capacity, `NVARCHAR` for identifiers.
- **Relationships:** PK `TableID`. Referenced by `Orders.TableID`.

5. Orders

- **Purpose:** The transactional hub of the system tracking the lifecycle of an order.
- **Important Columns:** `OrderType` (Dine-in/Takeaway/Campus Delivery), `OrderStatus`, `QueueNumber`, `EstimatedWaitingTime`, `TotalAmount`.
- **Data Types:** `DATETIME` for order time, `DECIMAL(10,2)` for totals.
- **Relationships:** PK `OrderID`, FK `UserID`, FK `TableID`.

6. OrderDetails

- **Purpose:** Represents the specific line items (food and drink) for a given order.
- **Important Columns:** `Quantity`, `UnitPrice`, `SubTotal`.
- **Data Types:** `INT` for quantities, `DECIMAL(10,2)` for currency.
- **Relationships:** PK `OrderDetailID`, FK `OrderID`, FK `MenuID`.

7. Payments

- **Purpose:** Records the financial settlement of an order.
- **Important Columns:** `PaymentMethod` (Cash/Card/Points), `Amount`, `PaymentStatus`.
- **Data Types:** `DECIMAL(10,2)` for amounts.
- **Relationships:** PK `PaymentID`, FK `OrderID` (UNIQUE).

4. Triggers Documentation

Triggers handle the automated business rules, shifting the processing burden from the application layer to the database engine.

1. trg_UpdateTableStatus_OnOrder

- **Purpose:** Automates table reservations.
- **When it executes:** AFTER INSERT on Orders.
- **Logic:** If an order is 'Dine-in' and has a TableID, it marks the corresponding table as 'Occupied'.
- **Example:** A student places a dine-in order and selects Table MH-01. The table instantly becomes unavailable to others.

2. trg_UpdateTableStatus_OnOrderComplete

- **Purpose:** Frees up tables automatically.
- **When it executes:** AFTER UPDATE on Orders.
- **Logic:** Checks if the QueueStatus or OrderStatus changes to 'Completed' or 'Cancelled'. If so, it reverts the linked table's status back to 'Available'.
- **Example:** Once the student finishes their meal and the order is marked 'Completed', Table MH-01 is free for the next group.

3. trg_CalculateTotalsAndWaitTime

- **Purpose:** Maintains accurate real-time financial and logistical totals.
- **When it executes:** AFTER INSERT, UPDATE, DELETE on OrderDetails.
- **Logic:** First calculates SubTotal (Quantity * UnitPrice) for the line item. Then, it aggregates all sub-totals and updates TotalAmount in the Orders table. It also sums the EstimatedPrepTime from the menu items and updates EstimatedWaitingTime.
- **Example:** Adding a pizza (15 mins, 100 LE) and a drink (5 mins, 20 LE) updates the order total to 120 LE and wait time to 20 mins.

4. trg_PreventInvalidData

- **Purpose:** Failsafe data validation.
- **When it executes:** AFTER INSERT, UPDATE on OrderDetails.
- **Logic:** Throws a fatal error and rolls back the transaction if a negative quantity or unit price is inserted.

5. trg_GenerateQueueNumber

- **Purpose:** Creates readable queue tickets for customers.
- **When it executes:** AFTER INSERT on Orders.
- **Logic:** Takes the newly generated OrderID and formats it into a 3-digit string prefixed with 'Q' (e.g., Order 12 becomes 'Q012').
- **Example:** Useful for the digital display boards in the food court.

6. trg_ProcessPaymentEffects

- **Purpose:** Triggers the kitchen workflow and loyalty rewards upon payment.
- **When it executes:** AFTER INSERT, UPDATE on Payments.
- **Logic:** If payment is 'Completed', the order's queue status shifts from 'Waiting' to 'Preparing'. It also calculates loyalty points (1 point per 10 LE spent) and updates the User's point balance.

5. Stored Procedures Documentation

Stored Procedures encapsulate complex business workflows into secure, reusable modules.

1. sp_ProcessFoodCourtOrder

- **Workflow:** Begins a transaction, inserts a master record into Orders, looks up current prices for up to 3 requested Menu Items, and inserts them into OrderDetails.
- **Parameters:** @UserID, @TableID, @OrderType, and up to 3 sets of (@MenuID, @Qty). Returns @NewOrderID via OUTPUT.
- **Validation:** Checks if items are currently 'Available'. Rolls back entirely if an item is missing.

2. sp_CampusDeliveryOrder

- **Workflow:** A shorthand wrapper for `sp_ProcessFoodCourtOrder` that forces `@OrderType` to 'Campus Delivery' and nullifies the `@TableID`.

3. sp_CompletePayment

- **Workflow:** Wraps payment logic in a transaction. Retrieves the final `TotalAmount` from the `Orders` table and inserts a new row into `Payments` with 'Completed' status.
- **Validation:** Ensures the order exists. Prevents double payments by checking if a payment record already exists.

4. sp_UpdateQueueStatus

- **Workflow:** Updates both `QueueStatus` and `OrderStatus` concurrently to ensure they remain synchronized.

5. sp_RedeemLoyaltyPoints

- **Workflow:** Deducts points from the user's account and applies a 1:1 monetary discount to the order's `TotalAmount`.
- **Validation:** Verifies the user has sufficient points before allowing the transaction.

6. sp_CancelOrder

- **Workflow:** Modifies both the `OrderStatus` and `QueueStatus` to 'Cancelled'. This subsequent update fires the table-freeing trigger.

6. Business Logic

- **Queue System:** Customers enter the queue as 'Waiting'. Once payment clears, the system automatically elevates them to 'Preparing'. Staff then manually update it to 'Ready' and finally 'Completed'. The auto-generated queue number acts as the primary customer-facing identifier.
- **Table Reservation Logic:** The system mirrors the real-world flow. A table is locked (**Occupied**) the moment a dine-in order is initiated. It remains locked until the customer finishes and the order is marked **Completed**, preventing double-booking.
- **Loyalty Points System:** Encourages ecosystem retention. Customers earn 10% back in points (1 point for every 10 currency units). Points act as a digital wallet that can be redeemed to subsidize future orders.
- **Payment Workflow:** Decoupled from order creation. An order sits in a 'Pending' financial state until **sp_CompletePayment** or **sp_RedeemLoyaltyPoints** is executed.
- **Order Processing Workflow:** **Order** -> **Order Details** -> (Triggers calculate Totals/Wait times) -> **Payment** -> (Triggers move to Preparing/Award Points).

7. Advanced Features

- **Automatic Calculations:** The database self-maintains financial aggregates. The application layer never manually updates **TotalAmount**; it relies entirely on **trg_CalculateTotalsAndWaitTime**.
- **Trigger Automation:** Business logic cascades naturally. A payment insert automatically affects the order queue and the user's loyalty profile.
- **Queue Number Generation:** Guarantees uniform, collision-free tracking numbers without application-side string manipulation.
- **Error Prevention:** **TRY...CATCH** blocks inside stored procedures handle exceptions gracefully. Check constraints and validation triggers block bad data at the schema level.
- **Cascading Deletes:** Deleting an **Order** automatically scrubs its associated **OrderDetails** and **Payments**, preventing orphan records.
- **Transactions:** Explicit **BEGIN TRANSACTION** and **COMMIT/ROLLBACK** guarantees atomicity. An order will never exist without its corresponding details if a mid-process failure occurs.

8. Sample Data Explanation

The database comes fully seeded with highly realistic, robust sample data.

- **Users:** 70 varied profiles (40 Students, 15 Professors, 15 Staff members), providing a realistic demographic mix.
- **Vendors:** 9 distinct food vendors. Includes realistic campus entities like "The Breakfast Bus", "Pasta Express", and newly integrated extensive menus for "Pablo", "Bites", and "Food Friends".
- **Menus:**
 - **Pablo:** Features pizzas, sandwiches, pastas, crepes, and waffles with different sizes and dynamic pricing.
 - **Bites:** A comprehensive cafe menu featuring diverse iced coffees, milkshakes, smoothies, bakery items, and savory sandwiches.
 - **Santa Cafe:** Focuses heavily on frappes, fresh juices, hot drinks, and customized croissants.
- **Sample Orders and Payments:** A complex T-SQL **WHILE** loop was utilized to simulate **150 unique, randomized orders**. The simulation randomly pairs users, vendors, items, and tables, processes payments (with a 90% completion rate and 10% cancellation rate), randomly allocates queue statuses, and simulates a 15% probability of loyalty point redemption.

9. SQL Queries Analysis

The system includes 15 advanced, business-intelligence queries to extract immediate value from the transactional data:

1. **Revenue Calculation (Most Profitable Vendor):** Aggregates sub-totals across completed orders to rank vendors by financial performance.
2. **Overall Order Statistics:** Calculates grand total revenue, average order value, and successful completion counts.
3. **Top Busiest Ordering Hours:** Uses `DATEPART` to analyze order timestamps and identify peak operational hours.
4. **Top Categories by Items Sold:** Groups by menu category to determine whether 'Pizza', 'Hot Drinks', or 'Sandwiches' are the highest volume drivers.
5. **Most Loyal Customers:** Identifies power-users based on their earned points.
6. **Orders by Type/Role:** Evaluates whether Students or Staff contribute more to the bottom line.
7. **Vendor Performance Ranking:** Utilizes Window Functions (`RANK() OVER`) to definitively rank vendors by revenue.
8. **Wait Time Analysis:** Calculates the average and maximum estimated wait times per vendor to identify bottlenecks.
9. **Cancellation Analysis:** Identifies lost revenue and calculates the exact percentage of orders that get cancelled.
10. **Payment Methods:** Aggregates transaction volumes by Cash, Card, and Online payments.
11. **Daily Revenue:** Groups payments by date to isolate the top 10 most profitable days.
12. **Vendor Pricing vs Volume:** Compares the average item price of a vendor against their total sales volume.
13. **Live Queue Dashboard:** A real-time `JOIN` query simulating a digital display board, showing current waiting/preparing orders.
14. **Top Expensive/Ordered Items:** Ranks the specific menu items that are absolute best-sellers across the entire food court.
15. **Table Utilization:** Assesses which tables/zones are most frequently occupied to optimize floor space.

10. Testing and Validation

- **How the system was tested:** A rigorous 150-order simulation script ([SECTION 6](#) in the SQL file) acts as a stress test. It programmatically navigates the entire order lifecycle, intentionally generating edge cases (like cancelled orders and randomized multi-item orders).
- **Example Test Cases:**
 - *Concurrency/Data Integrity:* Placing an order with 3 items simultaneously.
 - *Trigger Validation:* Completing a payment to verify if the queue status automatically shifts to 'Preparing' and points are awarded.
 - *Constraint Checking:* Attempting to book a 'Dine-in' order without a table or attempting to charge negative amounts.
- **Expected Outputs:**
 - The [TotalAmount](#) always perfectly matches the sum of [OrderDetails](#).
 - Tables are never double-booked.
 - Loyalty points are accurately mathematically tied to the completed [Payments](#) table.

11. Conclusion

Final System Summary

The University Food Court Management System is a highly automated, self-regulating database solution. By leveraging advanced T-SQL features like Triggers, Stored Procedures, and complex relational constraints, it abstracts complex business logic into the data layer itself.

Benefits of the System

- **Data Accuracy:** Automated calculations completely eliminate human math errors in pricing and wait times.
- **Seamless Operations:** Staff only need to interact with simple stored procedures to move orders through the pipeline.
- **Actionable Insights:** The comprehensive query suite allows university administration to make immediate, data-driven decisions regarding vendor leases, menu changes, and staffing during peak hours.

Future Improvements

- **Inventory Management:** Expanding the schema to track raw ingredients and map them to `MenuItems` for automated stock depletion.
- **Notification System:** Integrating a secondary table for push-notifications to alert users via SMS/Email when their order status hits 'Ready'.
- **Vendor Authentication:** Adding a vendor portal layer so individual vendors can toggle item `IsAvailable` statuses in real-time.